

Jakarta Persistence API - JPA

Khadim DRAME
kdrame@univ-zig.sn

Département Informatique
UFR Sciences et Technologies
Université Assane Seck de Ziguinchor

5 février 2025



Objectifs

- Créer des entités
- Utiliser les annotations (@Entity, @Table, @Id, @GeneratedValue,...)
- Réaliser les opérations CRUD avec **JPA** et **Hibernate**
- Créer des relations entre des entités **JPA**



Plan

- 1 Introduction
- 2 Entités
- 3 Opérations sur les entités
- 4 Langages de requêtes : JPQL et HQL
- 5 Relations entre entités



Avantages de JDBC

- Solution légère et portable
- Pratique pour des applications de petite taille



Introduction

Avantages de JDBC

- Solution légère et portable
- Pratique pour des applications de petite taille

Limites de JDBC

- Formulation des requêtes SQL
- Code SQL répétitif notamment pour les opérations CRUD
- Gestion de correspondances entre tables de la BD et objets



Introduction

Avantages de JDBC

- Solution légère et portable
- Pratique pour des applications de petite taille

Limites de JDBC

- Formulation des requêtes SQL
- Code SQL répétitif notamment pour les opérations CRUD
- Gestion de correspondances entre tables de la BD et objets

Solution : utiliser un **ORM**



Définition (Object Relational Mapping)

- Une couche d'abstraction qui permet de manipuler les tables d'une BD comme des objets
- Elle permet d'associer des classes aux tables d'une BD

Principe de mise en correspondance

Programme Java	Base de données
Classe	Table
Attribut d'une classe	Colonne d'une table
Référence entre classes	Relations entre tables
Objet	Ligne d'une table



- **Jakarta Persistence API** (depuis 2019) (anciennement **Java Persistence API**)
 - Elle n'est ni un **ORM** ni un **framework**
 - C'est une **spécification**, un ensemble d'interfaces standardisé par **Oracle** pour gérer la persistance des données
 - Les frameworks ORM doivent fournir une implémentation de cette spécification
-
- JPA s'appuie sur JDBC pour communiquer avec une BD
 - Les correspondances peuvent se faire via des **annotations JPA** dans les classes ou des fichiers de configuration



Comme pour JDBC, l'utilisation de JPA nécessite un fournisseur de persistance qui implémente l'API

Frameworks ORM implémentant les spécifications JPA

- **Hibernate** (Red Hat)
- EclipseLink (Eclipse Foundation)
- TopLink (Oracle)
- Apache OpenJPA
- Java Data Objects (JDO)
- ...



- Framework ORM développé depuis mai 2001, open-source et écrit en Java
- Il peut être utilisé dans tout type de projet (Console, Web,)
- Hibernate disposait de ses propres interfaces, classes, méthodes et annotations pour la persistance de données : **Hibernate Native API**
- La plupart des opérations sont réalisables avec des méthodes natives de Hibernate



Plan

- 1 Introduction
- 2 Entités**
- 3 Opérations sur les entités
- 4 Langages de requêtes : JPQL et HQL
- 5 Relations entre entités



- Une **entité** est une classe permettant d'encapsuler les données d'une ou de plusieurs tables
- Les entités sont de simples classes appelées **POJO** (**Plain Old Java Object**) :
 - un constructeur public sans paramètre (constructeur par défaut)
 - possibilité de contenir d'autres constructeurs
 - toutes les propriétés sont **privées**
 - un *getter* et *setter* publics pour chaque attribut



Les entités

Exemple (POJO)

```
1 package sn.uasz.m1.models;
2 public class Personne {
3     private long id;
4     private String prenom;
5     private String nom;
6     private int age;
7     public Personne(){}
8     public int getId() { return id;}
9     public void setId(long id) { this.id = id;}
10    public String getPrenom() { return prenom;}
11    public void setPrenom(String prenom) { this.prenom = prenom;}
12    public String getNom() { return nom;}
13    public void setNom(String nom) { this.nom = nom;}
14    //setter et getter de l'attribut age
15    public String toString(){ return id+"-"+prenom+" "+nom; }
16 }
```



Définition (Entité)

- Une **entité** (**Entity**) est un POJO mappé vers une table de la base de données grâce à des métadonnées via JPA
- Entité = POJO + métadonnées permettant le mapping
- Elle ne peut pas être une interface ou une énumération

Deux solutions pour le mapping

- soit un fichier xml de mapping nommé `EntityName.hbm.xml`
- soit des **annotations JPA** : **@Entity**, **@Id**,...



Les annotations

Définition (annotation)

- Une **annotation** est une description d'un élément constituée d'un mot-clé précédée du signe **@**
- Elle peut s'appliquer à une classe, une méthode, un attribut, ...

Annotations pour le mapping objet-relationnel

- **@Entity** : indique que la classe est persistante
- **@Table** : permet de mapper l'entité à une table de la BD. Par défaut, l'entité est mappée à la table de la BD de même nom
- **@Id** : déclare un attribut de l'entité comme étant la clé primaire de la table correspondante



Annotations pour le mapping objet-relationnel

- **@Column** : permet d'associer un attribut de l'entité à une colonne de la table
- **@GeneratedValue** : permet de générer automatiquement la clé primaire de la table annotée par @Id
- **@IdClass** : indique que l'entité contient une clé composée
- **@Convert** : indique comment convertir la valeur avant de la persister



Les annotations

Exemple

```
1 package sn.uasz.m1.models;
2
3 @Entity
4 public class Personne {
5     @Id
6     @GeneratedValue(strategy=GenerationType.IDENTITY)
7     private long id;
8
9     @Column(name="first_name", length=30)
10    private String prenom;
11
12    @Column(name="family_name", length=20)
13    private String nom;
14
15    private int age;
16
17    // reste du code
18 }
```

Plan

- 1 Introduction
- 2 Entités
- 3 Opérations sur les entités**
- 4 Langages de requêtes : JPQL et HQL
- 5 Relations entre entités



Les opérations sur les entités

- Pour rendre les données persistantes, on utilise le gestionnaire d'entités : `EntityManager`
- Toute entité JPA persistante possède une référence sur l'`EntityManager` qui la créait ou qui a permis de la retrouver
- Les opérations de création, de suppression ou de modification doivent nécessairement se faire dans une transaction
- Démarrage d'une transaction : `begin()`, validation de la transaction : `commit()`, annulation d'une transaction : `rollback()`



Configuration avec Persistence.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence xmlns="http://java.sun.com/xml/ns/persistence" xmlns:xsi="http://www.w3.org
  /2001/XMLSchema-instance" xsi:schemaLocation="http://java.sun.com/xml/ns/
  persistence http://java.sun.com/xml/ns/persistence/persistence_2_0.xsd" version="
  2.0">
3 <persistence-unit name="demojpa-pu">
4 <!-- <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider> -->
5 <description>Test de Hibernate JPA </description>
6 <class>sn.uas.model.Personne</class>
7 <exclude-unlisted-classes>true</exclude-unlisted-classes>
8 <properties>
9 <property name="hibernate.dialect" value="org.hibernate.dialect.MySQL8Dialect" />
10 <property name="hibernate.hbm2ddl.auto" value="update" />
11 <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>
12 <property name="jakarta.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/club
  ?serverTimezone=GMT" />
13 <property name="jakarta.persistence.jdbc.user" value="root" />
14 <property name="jakarta.persistence.jdbc.password" value="" />
15 <property name="hibernate.format_sql" value="true" />
16 <property name="hibernate.show_sql" value="true" />
17 </properties>
18 </persistence-unit>
19 </persistence>
```

Configuration avec Persistence.xml

- Pour autoriser Hibernate à créer la BD si elle n'existe pas

```
1 <property name="jakarta.persistence.jdbc.url" value="
    jdbc:mysql://localhost:3306/club?createDatabaseIfNotExist=
    true?serverTimezone=GMT" />
```

- La propriété **hbm2ddl.auto** peut prendre comme valeur
 - **create** : crée les tables, les données précédemment présentes dans les tables seront perdues
 - **update** : met à jour les tables avec les valeurs données. Si les tables n'existent pas dans la BD, elles seront créées
 - **validate** : si les tables n'existent pas dans la BD, elles ne seront pas créées et une exception sera levée



CRUD avec JPA

Insertion : persist()

```
1 //Ouverture d'une unité de travail JPA
2 EntityManagerFactory emf = Persistence.createEntityManagerFactory(
    "demojpa-pu");
3 // 1 : Création d'un gestionnaire d'entité
4 EntityManager em = emf.createEntityManager();
5 // 2 : Démarrage de la transaction
6 EntityTransaction transaction = em.getTransaction();
7 transaction.begin();
8 // 3 : Instanciation d'un objet métier
9 Personne personne = new Personne("Nafissatou", "Thiam", 35);
10 // 4 : Insertion de l'objet dans la BD
11 em.persist(personne);
12 // 5 : Fermeture de la transaction
13 transaction.commit();
14 // 6 : Fermeture des flux
15 em.close();
16 emf.close();
```

Recherche selon l'identifiant : find()/getReference()

```
1 EntityManagerFactory emf = Persistence.createEntityManagerFactory(  
    "demojpa-pu");  
2 EntityManager em = emf.createEntityManager();  
3 EntityTransaction transaction = em.getTransaction();  
4 transaction.begin();  
5  
6 // 3 : Recherche d'un objet selon son identifiant  
7 Personne personne = em.find(Personne.class, 1);  
8 System.out.println(personne);  
9  
10 transaction.commit();  
11 em.close();  
12 emf.close();
```



Modification d'un objet

```
1 EntityManagerFactory emf = Persistence.createEntityManagerFactory(  
    "demojpa-pu");  
2 EntityManager em = emf.createEntityManager();  
3 EntityTransaction transaction = em.getTransaction();  
4 transaction.begin();  
5  
6 // Recherche d'un objet selon son identifiant  
7 Personne personne = em.find(Personne.class, 1);  
8 // Modification de l'objet  
9 personne.setPrenom("Maryama");  
10 em.flush(); // ou em.persist(personne);  
11  
12 transaction.commit();  
13 em.close();  
14 emf.close();
```



Suppression d'un objet : remove()

```
1 EntityManagerFactory emf = Persistence.createEntityManagerFactory(  
    "demojpa-pu");  
2 EntityManager em = emf.createEntityManager();  
3 EntityTransaction transaction = em.getTransaction();  
4 transaction.begin();  
5  
6 // Recherche d'un objet selon son identifiant  
7 Personne personne = em.find(Personne.class, 1);  
8 // Suppression de l'objet  
9 em.remove(personne);  
10  
11 transaction.commit();  
12 em.close();  
13 emf.close();
```



Plan

- 1 Introduction
- 2 Entités
- 3 Opérations sur les entités
- 4 Langages de requêtes : JPQL et HQL
- 5 Relations entre entités



JPA et **Hibernate** fournissent des langages de requêtes permettant de manipuler les entités : **JPQL** (**JPA Query Language**) et **HQL** (**Hibernate Query Language**)

HQL

- Langage de requêtes pour les entités **JPA** définies par Hibernate
- Inspiré du langage **SQL**
- Il permet de manipuler les entités et pas les tables



HQL

- Il supporte les requêtes de type SELECT, INSERT, UPDATE et DELETE
- Seule la clause FROM est obligatoire dans une requête SELECT

Méthodes pour l'exécution de requêtes

- `createNativeQuery()` : permet d'exécuter une requête **SQL**
- `createNamedQuery()` : permet d'exécuter une requête **HQL** définie par des annotations
- `createQuery()` : permet d'exécuter une requête **HQL**



Exemple

```
1 package sn.uaszi.l3ii.model;  
2 public class DemoHQL {  
3  
4     public static void main(String[] args) {  
5         EntityManagerFactory emf = Persistence.createEntityManagerFactory("demojpa-pu");  
6         EntityManager em = emf.createEntityManager();  
7  
8         String req = "FROM Personne p WHERE p.nom = :nom";  
9         TypedQuery<Personne> query = em.createQuery(req, Personne.class);  
10        String nom = "DIALLO";  
11        query.setParameter("nom", nom);  
12  
13        List<Personne> personnes = query.getResultList();  
14        //personnes.forEach(System.out::println);  
15        for( Personne p : personnes)  
16            System.out.println(p);  
17  
18        em.close();  
19        emf.close();  
20    }  
21  
22 }
```



Plan

- 1 Introduction
- 2 Entités
- 3 Opérations sur les entités
- 4 Langages de requêtes : JPQL et HQL
- 5 Relations entre entités**



Relations entre entités JPA

Quatre types de relations possibles entre entités JPA

- Relation **1:1** : annotée par @OneToOne
- Relation **n:1** : annotée par @ManyToOne
- Relation **1:p** : annotée par @OneToMany
- Relation **n:p** : annotée par @ManyToMany



Relation 1:1

- On considère qu'un employé a une et une seule adresse
- On ajoute l'association **@OneToOne** dans Employe

```
1 @Entity
2 public class Employe {
3     @Id
4     @GeneratedValue(strategy =
5         GenerationType.IDENTITY
6     )
7     private int id;
8     private String nom;
9     private double salaire;
10    @OneToOne
11    private Adresse adresse;
12    // suite de la classe
13 }
```

```
1 @Entity
2 public class Adresse {
3     @Id
4     @GeneratedValue(strategy =
5         GenerationType.IDENTITY
6     )
7     private int id;
8     private String rue;
9     private String ville;
10    // suite de la classe
11 }
```

- Employe est l'entité **maître** et Adresse l'entité **esclave**



Relation 1:1

- L'annotation **@JoinColumn** permet de spécifier plus de détails sur la colonne de jointure

```
1 @OneToOne
2 @JoinColumn(name="id_adresse", referencedColumnName="id",
3             nullable=false)
4 private Adresse adresse;
```

- Pour ajouter un employé et son adresse et les rendre persistantes

```
1 Adresse adresse = new Adresse("Rue Javelier", "Ziguinchor");
2 Employe emp = new Employe("Joseph Sarr", 200000, adresse);
3 em.persist(adresse);
4 em.persist(emp);
```



Relation 1:1

- L'annotation **@JoinColumn** permet de spécifier plus de détails sur la colonne de jointure

```
1 @OneToOne
2 @JoinColumn(name="id_adresse", referencedColumnName="id",
3             nullable=false)
4 private Adresse adresse;
```

- Pour ajouter un employé et son adresse et les rendre persistantes

```
1 Adresse adresse = new Adresse("Rue Javelier", "Ziguinchor");
2 Employe emp = new Employe("Joseph Sarr", 200000, adresse);
3 em.persist(adresse);
4 em.persist(emp);
```

- Deux tables créées : Adresse et Employe, deux tuples insérés



Opérations en cascade

- On peut utiliser l'attribut **cascade** dans l'annotation @OneToOne

```
1 @Entity
2 public class Employe {
3     // ...
4     @OneToOne(cascade = {CascadeType.PERSIST})
5     private Adresse adresse;
6     // ...
7 }
```

- On peut ainsi faire un seul appel de persist()

```
1 Adresse adresse = new Adresse("Rue Javelier", "Ziguinchor");
2 Employe emp = new Employe("Joseph Sarr", 200000, adresse);
3 em.persist(emp);
```

- On peut cascader d'autres opérations telles que REMOVE ou toutes (ALL)



Relation n:1

- On considère que plusieurs employés peuvent avoir la même adresse
- On ajoute l'association **@ManyToOne** dans Employe

```
1 @Entity
2 public class Employe {
3     @Id
4     @GeneratedValue(strategy =
5         GenerationType.IDENTITY
6     )
7     private int id;
8     private String nom;
9     private double salaire;
10    @ManyToOne
11    private Adresse adresse;
12    // suite de la classe
13 }
```

```
1 @Entity
2 public class Adresse {
3     @Id
4     @GeneratedValue(strategy =
5         GenerationType.IDENTITY
6     )
7     private int id;
8     private String rue;
9     private String ville;
10    // suite de la classe
11 }
```

- NB : Le schéma de la BD ne change pas !



Relation n:1

- Pour ajouter une adresse et deux employés et les rendre persistantes

```
1 Adresse adresse = new Adresse("Rue Javelier", "Ziguinchor");  
2 Employe emp1 = new Employe("Joseph Sarr", 250000, adresse);  
3 Employe emp2 = new Employe("Fatou Faye", 300000, adresse);  
4 em.persist(emp1);  
5 em.persist(emp2);
```



Relation n:1

- Pour ajouter une adresse et deux employés et les rendre persistantes

```
1 Adresse adresse = new Adresse("Rue Javelier", "Ziguinchor");  
2 Employe emp1 = new Employe("Joseph Sarr", 250000, adresse);  
3 Employe emp2 = new Employe("Fatou Faye", 300000, adresse);  
4 em.persist(emp1);  
5 em.persist(emp2);
```

- Deux tables créées : Adresse et Employe, trois tuples insérés



Relation 1:n

- On considère qu'une entreprise peut avoir plusieurs employés
- On ajoute l'association **@OneToMany** dans Entreprise

```
1 @Entity
2 public class Entreprise {
3     @Id
4     @GeneratedValue(strategy =
5         GenerationType.AUTO)
6     private int code;
7     private String nom;
8     private double capital;
9     @OneToMany(cascade = {
10         CascadeType.PERSIST})
11     private List<Employe>
12         employes;
13     // suite de la classe
14 }
```

```
1 @Entity
2 public class Employe {
3     @Id
4     @GeneratedValue(strategy =
5         GenerationType.IDENTITY
6     )
7     private int id;
8     private String nom;
9     private double salaire;
10     // suite de la classe
11 }
```



Relation 1:n

- Définir le getter et le setter de employes
- Ajouter les méthodes addEmploye() et removeEmploye() permettant d'ajouter ou de supprimer un employé dans une entreprise

```
1 Adresse adresse = new Adresse("Rue Javelier", "Ziguinchor");
2 Employe emp1 = new Employe("Joseph Sarr", 250000, adresse);
3 Employe emp2 = new Employe("Fatou Faye", 300000, adresse);
4 Entreprise entreprise = new Entreprise("Casamançaise",
    1700000);
5 entreprise.addEmploye(emp1);
6 entreprise.addEmploye(emp2);
7 em.persist(entreprise);
```



Relation 1:n

- Définir le getter et le setter de employes
- Ajouter les méthodes addEmploye() et removeEmploye() permettant d'ajouter ou de supprimer un employé dans une entreprise

```
1 Adresse adresse = new Adresse("Rue Javelier", "Ziguinchor");
2 Employe emp1 = new Employe("Joseph Sarr", 250000, adresse);
3 Employe emp2 = new Employe("Fatou Faye", 300000, adresse);
4 Entreprise entreprise = new Entreprise("Casamançaise",
    1700000);
5 entreprise.addEmploye(emp1);
6 entreprise.addEmploye(emp2);
7 em.persist(entreprise);
```

- Quatre tables créées : Adresse, Employe, Entreprise et Entreprise_Employe
- Six tuples insérés : une adresse, 2 employés, une entreprise et 2 dans Entreprise_Employe



Exemple

- Une personne peut pratiquer plusieurs sports
 - Un sport peut être pratiqué par plusieurs personnes
-
- Dans ce cas, on peut utiliser l'annotation **@ManyToMany** dans une des entités : par exemple dans **Personne**
 - Définir le getter et le setter de la *collection* de sports
 - Ajouter les méthodes `addSport()` et `removeSport()` permettant d'ajouter ou de retirer un sport de la liste de sports pratiqués par la personne



Relation n:p

```
1 @Entity
2 public class Sport {
3
4     @Id
5     @GeneratedValue (strategy=
6         GenerationType.AUTO)
7     private long code;
8     @Column(length = 40)
9     private String nom;
10    private String type;
11
12    // reste du code
13 }
```

```
1 @Entity
2 public class Personne {
3
4     @Id
5     @GeneratedValue (strategy=
6         GenerationType.IDENTITY)
7     private int id;
8     private String nom;
9
10    @ManyToMany(cascade = {
11        CascadeType.PERSIST })
12    private List<Sport> sports
13    // suite de la classe
14 }
```



Relation n:p

```
1 Personne personne1 = new Personne("Ronaldo");
2 Personne personne2 = new Personne("Jordan");
3 Sport sport1 = new Sport("Football", "collectif");
4 Sport sport2 = new Sport("Tennis", "individuel");
5 Sport sport3 = new Sport("Basketball", "collectif");
6 personne1.addSport(sport1);
7 personne1.addSport(sport2);
8 personne2.addSport(sport3);
9 em.persist(personne1);
10 em.persist(personne2);
```



Relation n:p

```
1 Personne personne1 = new Personne("Ronaldo");
2 Personne personne2 = new Personne("Jordan");
3 Sport sport1 = new Sport("Football", "collectif");
4 Sport sport2 = new Sport("Tennis", "individuel");
5 Sport sport3 = new Sport("Basketball", "collecif");
6 personne1.addSport(sport1);
7 personne1.addSport(sport2);
8 personne2.addSport(sport3);
9 em.persist(personne1);
10 em.persist(personne2);
```

- Trois tables créées : Sport, Personne et Personne_Sport
- Huit tuples insérés : 2 personnes, 3 sports, et 3 dans Personne_Sport

